Causal Explanations of Process Monitor Predictions

Tom Yaacov
Dept. of Informatics
King's College London
London, UK
tom.yaacov@kcl.ac.uk

Nathan Blake
Dept. of Informatics
King's College London
London, UK
nathan.blake@kcl.ac.uk

Hana Chockler
Dept. of Informatics
King's College London
London, UK
hana.chockler@kcl.ac.uk

***Abstract*—Process mining is widely used to diagnose processes and identify performance and compliance issues. Specifically, Predictive Process Monitoring (PPM) techniques use AI models to predict outcomes of ongoing process instances. While these models can achieve high predictive performance, their black-box nature makes it difficult to understand the underlying reasons behind their output predictions. In this paper, we propose a novel approach for generating local (case-level) explanations of process monitor predictions based on the framework of actual causality. We define a causal model tailored to processes that captures temporal dependencies between events in a trace, thus allowing us to reason about causal influence of events on the predicted outcome. Our method uses this model implicitly to compute causes and quantify the importance of different events with respect to the predicted outcome. We present a practical, model-agnostic algorithm that approximates event responsibility given the process structure reflected in the causal model. We evaluate our approach on a range of datasets derived from real-life event logs from a standard PPM benchmark. Each dataset contains up to 130,000 traces, with trace lengths of up to 1,800 events and up to 400 distinct event types. We compare our approach with state-of-the-art local explanation methods. The results demonstrate that our approach produces more stable and concise explanations while maintaining competitive efficiency.**

***Index Terms*—Predictive Process Monitoring, Explainable Artificial Intelligence, Actual Causality, Interpretability, Predictive Process Analytics, Business Process Management, Log Files Analysis**

I. Introduction

Process mining techniques analyze event logs from an information system in order to discover, monitor, and ultimately improve the process. Predictive Process Monitoring (PPM) is a subfield of process mining that applies machine learning methods to predict future events in the process. Deep learning (DL) has substantially improved this predictive capability [1]. However, DL models are typically opaque, giving no insights into how a prediction was made. Explainable AI (XAI) refers to a family of techniques designed to confer some understanding of the workings of DL models, facilitating trust in their predictions [2]. Post-hoc XAI is one type of XAI which is applied to a model after training and explains a particular instance. These are by far the most commonly applied to PPM, with Shapley Additive exPlanations (SHAP) [3] and Local Interpretable Model-Agnostic Explanations (LIME) [4] being the most popular methods [5]. However, these are general XAI methods, developed for imaging and tabular data, and do not account for the temporal and causal structures inherent to event logs. They are particularly ill-suited to tasks involving highly correlated datasets [6], which is common in PPM domains such as healthcare where understanding causal relationships is crucial. The fidelity of XAI techniques is important in any business process, whether this be for profit maximization, or a medico-ethical imperative in fields such as medicine. These necessitate more refined XAI approaches which capture causal structures.

In this proof-of-concept work, we seek to remedy these limitations by using the framework of *actual causality*, introduced by Halpern and Pearl [7] and extended by Halpern in [8]. We introduce a causal model for processes that captures temporal dependencies between events in a trace and enables reasoning about their causal influence on process monitor predictions. Using this model, we derive explanations in terms of causes and quantify the importance of different events using the established notion of causal *degree of responsibility*. We further present a model-agnostic algorithm that approximates the causal responsibility of events in the trace for predictions. This algorithm is implemented in the AC4PM tool (Actual Causality for Process Monitoring), which we compare with LIME and SHAP.

The framework of actual causality is different to *type causality* as introduced by Pearl [9], which deduces causal relationships between the variables based on the existing data, and is forward-looking, that is, used for prediction. In contrast, the study of actual causality is backward-looking ("what caused a certain event to happen?"), in alignment with our interest in this paper. The reader is referred to Sec. II-B for an overview of actual causality.

Counterfactual methods have been introduced to address concerns similar to those we consider in this paper [10]. These XAI methods aim to identify minimal changes to a process instance that would alter the prediction outcome, thereby providing actionable insights for process improvement. This gives practitioners access to 'what-if' examples which are intuitive to grasp. However, certain causal pathways can be particularly nuanced and existing counterfactual methods do not capture all of these. We consider four causal behaviors which counterfactuals do not fully capture: *over-determination*, *preemption*, *non-minimal counterfactuals* and *causally inconsistent counterfactuals*.

Preemption occurs when a cause first produces an outcome, preventing another potential cause from achieving the same

effect. As an example from medicine, consider a patient who receives a blood transfusion. They would die from hyperkalaemia but this is preempted in this case by first dying from an anaphylactic reaction (see Fig. 1). A counterfactual explanation might fail to capture that it is the anaphylactic reaction that caused the death, since if the patient had not had the reaction, they would still have died.

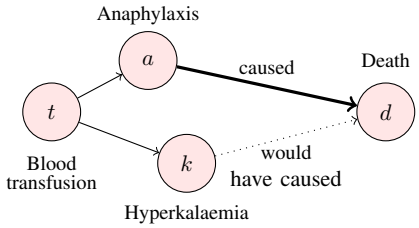

Fig. 1: Illustration of preemption. Blood transfusion (t) triggers an anaphylactic reaction (a) that causes death (d). Hyperkalaemia (high serum potassium) would also have caused death, but this alternative pathway was *preempted* by death from anaphylaxis.

Over-determination occurs when multiple *independent* causes exist, any of which on their own are sufficient to an outcome. Consider a patient in a road traffic accident: their catastrophic head injury would be sufficient to lead to death, but so too would a ruptured aorta (see Fig. 2). A counterfactual explanation might suggest that the two together led to death, without adequately capturing that each alone is sufficient to cause death. This differs from preemption in the temporal ordering of events: in preemption a preceding event brings about an outcome, preempting another event from occurring. In over-determination both sufficient independent events occur simultaneously.

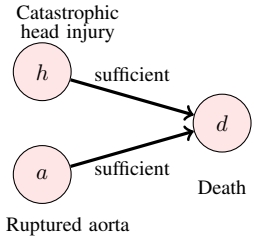

Fig. 2: An example of over-determination. Road traffic accident causes both a catastrophic head injury (h) and a ruptured aorta (a). Either injury alone is sufficient to cause death (d), so the outcome is *over-determined*.

Causally inconsistent counterfactuals occur when a counterfactual is applied which successfully flips a predicted outcome, but violates the underlying causal structure (see Fig. 3). Improved counterfactual methods generally capture this dependence [11]. Non-minimal counterfactuals occur when changes are made which are not part of the cause. This successfully changes the outcome, but obfuscates the actual causal path that was followed (see Fig. 4).

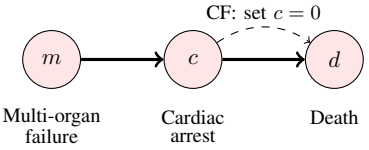

Fig. 3: An example of a causally-inconsistent counterfactual (CF): a particular patient has multi-organ failure (m), which in this case causes cardiac arrest, which causes death (d). A counterfactual that sets $c = 0$, while leaving m unchanged, may flip the outcome prediction, but it violates the underlying causal structure because the cardiac arrest is itself caused by the organ failure.

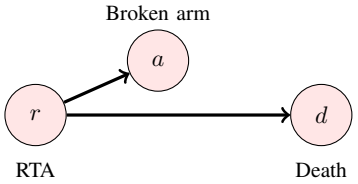

Fig. 4: An example of a non-minimal counterfactual: a road-traffic accident (RTA) (r) causes both broken arm (a) and death (d). Clearly, a broken arm is insufficient to cause death. However, the counterfactual trace, $\langle r = 0, a = 0, d = 0 \rangle$, would not be able to discriminate between r and a as the actual cause.

Actual causality extends and generalizes counterfactual (but-for) causality by allowing structured counterfactual reasoning under *contingencies* in causal models. In this setting, an event A is considered an actual cause of B if, possibly under some contingency on other variables, B counterfactually depends on A. This framework can accurately capture both preemption and over-determination. Moreover, by restricting attention to plausible contingencies within the causal model, it avoids explanations that rely on inconsistent or unrealistic scenarios. Actual causality also allows reasoning about particular instances in the causal model, which helps address issues of non-minimal counterfactual explanations.

We evaluate AC4PM on 22 datasets derived from nine real-life event logs from a widely used predictive process monitoring benchmark. The datasets vary substantially in size and complexity, containing up to 130,000 traces, with trace lengths of up to 1,800 events and up to 400 distinct event types. We compare AC4PM against LIME and SHAP using established evaluation criteria for explainability, including consistency, succinctness, and computational efficiency. The results show that AC4PM produces explanations that are generally more stable and concise than those of the competing approaches, while remaining computationally practical across all evaluated benchmarks.

The paper structure is as follows. Sec. II introduces the necessary background on process mining and the framework of actual causality. Sec. III presents the proposed causal model for processes and discusses the design principles underly-

ing it. Sec. IV describes our algorithm for approximating the causal responsibility of events in a trace with respect to the prediction of a process monitor. Sec. V details an experimental evaluation of the suggested approach on a range of PPM benchmarks, and compares it with state-of-the-art explanation methods. We review work related to this paper in Sec. VI and conclude in Sec. VII. The full evaluation results and code are available in the supplementary material at https://figshare.com/s/7d9602ca44991641c56a.

II. Preliminaries

A. Process Mining and Predictive Process Monitoring

Process mining comprises a range of techniques for analyzing event data to understand and improve operational processes. In what follows, we briefly present the relevant definition from process mining.

We assume a process instance is a sequence of *activities* taken from a finite set $\mathcal{A}$, allowed in the context of a given business process. An *event* corresponds to the occurrence of an activity as a part of a specific process instance, called a *case*. Events may include additional attributes, such as a timestamp or resources. In this work, we consider a simplified representation where only activity names are recorded for each event, and the timestamps are only used for event ordering. This means that each case can be represented as a sequence of activities, called a *trace* $\sigma = \langle e_1, e_2, \ldots, e_n \rangle \in \mathcal{A}^*$, where each event $e_i \in \mathcal{A}$ marks the activity executed at position i in the trace. An *event log* is a collection of traces representing multiple executions of the same process. More formally, an event log is a multiset (since a log can contain multiple instances of the same trace) of traces.

Predictive Process Monitoring (PPM) is a subfield of process mining that aims to predict future aspects of an ongoing business process execution. Examples of prediction targets include the outcome of a process instance, its completion, or its future activities. In this paper, we focus on categorical prediction targets, and therefore assume that the process monitor is a classifier, denoted by $\mathcal{C}$. PPM typically consists of two phases: an offline phase, in which a collection of labeled traces (or prefixes of traces) is used to train $\mathcal{C}$, and an online phase, in which, given a trace (or a prefix of it), $\mathcal{C}$ predicts its label. A comprehensive overview of predictive process monitoring can be found in [12].

B. Actual Causality

In what follows, we informally introduce the relevant concepts from the theory of actual causality. The reader is referred to [8] for a more in-depth overview.

We assume the world is described in terms of variables and their values. Variables may have a causal influence on others. It is useful to split these variables into two sets: the *exogenous* variables, $\mathcal{U}$, whose values are determined by factors outside the causal model, and the *endogenous* variables, $\mathcal{V}$, whose values are dependent on and ultimately determined by the values of exogenous variables.

Formally, a *causal model* is a pair $M = (\mathcal{S}, \mathcal{F})$, where $\mathcal{S} = (\mathcal{U}, \mathcal{V}, \mathcal{R})$ is a *signature*, which lists the set of exogenous variables $\mathcal{U}$ and endogenous variables $\mathcal{V}$. $\mathcal{R}$ is a mapping which associates every variable $Y \in \mathcal{U} \cup \mathcal{V}$ with a non-empty set $\mathcal{R}(Y)$ of possible domain values for Y. $\mathcal{F}$ defines a set of structural equations that determine the values of each endogenous variable using variables in $\mathcal{U} \cup \mathcal{V}$.

A *context* $\vec{u}$ is a setting for the exogenous variables $\mathcal{U}$. We call a pair $(M, \vec{u})$ consisting of a causal model M and a context $\vec{u}$, a *causal setting*. $\vec{u}$ determines, through the structural equations, the values of all endogenous variables. This can be either directly or indirectly, when an endogenous variable depends on other endogenous variables. Following previous literature, we assume that M is recursive (acyclic). That means that given a context $\vec{u}$, the values of all other variables are determined. We write $(M, \vec{u}) \models \varphi$ if a Boolean formula φ is true in the causal setting $(M, \vec{u})$.

A causal model allows us to perform *interventions* on $(M, \vec{u})$ by changing the value of some variable(s) (subset of $\mathcal{V}$) X to x, which essentially amounts to replacing the equation of X in $\mathcal{F}$ to $X = x$. We denote it by $[X \leftarrow x]$. With that, we can define actual causes:

Definition 1 (Actual cause [8], [13]): $\vec{X} = \vec{x}$, a subset of the endogenous variables and their valuation, is an *actual cause* of φ in the causal setting $(M, \vec{u})$ if the following three conditions hold:

AC1. $(M, \vec{u}) \models (\vec{X} = \vec{x})$ and $(M, \vec{u}) \models \varphi$.

AC2. There is an alternative setting $\vec{x}'$ of the variables in $\vec{X}$, a (possibly empty) set $\vec{W}$ of variables in $\mathcal{V} - \vec{X}$, and a setting $\vec{w}$ of the variables in $\vec{W}$ such that $(M, \vec{u}) \models \vec{W} = \vec{w}$ and $(M, \vec{u})[\vec{X} \leftarrow \vec{x}', \vec{W} \leftarrow \vec{w}] \models \neg\varphi$.

AC3. $\vec{X}$ is minimal, i.e., there is no strict subset $\vec{X}'$ of $\vec{X}$ such that $\vec{X}' = \vec{x}''$ can replace $\vec{X} = \vec{x}'$ in AC2, where $\vec{x}''$ is the restriction of $\vec{x}'$ to the variables in $\vec{X}'$.

Essentialy, AC1 states that the cause, $\vec{X} = \vec{x}$, and the outcome, φ indeed happened in context $\vec{u}$ (this is *actual* causality). AC2 says that for $\vec{X} = \vec{x}$ to be a cause of φ, there must be an assignment $\vec{x}'$ for $\vec{X}$ such that if we clamp $\vec{X} = \vec{x}'$ and $\vec{W} = \vec{w}$, φ no longer holds. AC3 states that there should be no unnecessary variables in $\vec{X} = \vec{x}$. A variable X in an actual cause $\vec{X}$ is called a *part of a cause*. Following [13], we often refer to parts of causes as causes.

In this paper, we use the definition of *degree of responsibility*, first introduced in [14], which quantifies the measure of causal influence.

Definition 2 (Responsibility [14]): The *degree of responsibility* of an endogenous variable X for φ in $(M, \vec{u})$ is defined as $1/(|\vec{X}| + |\vec{W}|)$, where $\vec{X}$ is a smallest actual cause for φ that contains X, and $\vec{W}$ is its respective contingency. If there is no actual cause containing X, its degree of responsibility is 0.

In the context of explaining the predictions of black-box models, the definition of responsibility provides an ordering of features based on their importance to the predicted value.

III. Causal Model for Processes

We now present our general causal model for processes. Given a process P and its predictive process monitor, $\mathcal{C}$, we define a causal model $M_{P,\mathcal{C}}$ as follows (see Fig. 5). The set $\mathcal{V} = \vec{V} \cup \{O\}$ of endogenous variables consists of a set $\vec{V}$ corresponding to the trace events, $e_1, \ldots, e_n$, where e_i is the event in step i. O is the output variable, indicating the output of $\mathcal{C}$ over the trace induced by the assignment of $\vec{V}$. The set $\vec{U}$ of exogenous variables captures external factors and sources of randomness that influence the execution of the process. Consequently, once the exogenous variables are assigned with a context $\vec{U} = \vec{u}$, the resulting trace, and therefore the values of $\vec{V}$, can be fully determined.

The general structure selected for the causal model maintains several key principles in the context of causal analysis of processes:

Temporal aspect: Previous work that applied the framework of actual causality to explain black-box AI systems considered a depth-2 causal model [15], [16], similar in structure to that of Fig. 5, except that all variables in $\vec{V}$ are independent of each other. While this assumption is reasonable in other cases and might help in finding causes more efficiently [16], this is not suitable for the case of processes where intervening on one event may affect subsequent events. To capture this temporal dependency, we model each event as causally dependent on its preceding events.

Natural ordering: The value of the event e_i as depicted in Fig. 5 causally depends on the values of the preceding events $e_1, \ldots, e_{i-1}$. We note that this might not always be the case. For instance, e_3 might only depend on e_1 but not on e_2 for some process. The causal model allows the dependencies on all preceding events, as this is the most general setting. The order of causal dependence is determined by the temporal order of the events.

Reachability: In our setting, we only consider interventions that lead to traces that are consistent with the underlying behavior of the process (see Fig. 3 illustrates a possible inconsistent intervention). While in the theory of actual causality interventions that disrupt the causal dependencies are allowed (in fact, this is exactly what interventions are), since our only source of information about the causal model is the set of traces, we restrict it to consistent (or *reachable*) traces. This aligns with the notion of *normality* in actual causality, in which we only consider possible alternatives that can be induced by another context [17]. More specifically, process P defines a set of possible contexts (values of exogenous variables) for the causal model. Each context leads to an actual trace that P can produce. As for the process monitor, we evaluate interventions that remain within the distribution on which it was trained.

Simplicity: The size of the model depends on the length of the trace. However, as we show in Sec. IV, its simple structure allows us to approximate causes and responsibilities without explicitly constructing the full causal model. Furthermore, since each event variable depends on all previous events, we can represent the process using only the event variables themselves, without introducing additional auxiliary variables to keep track of the state during execution.

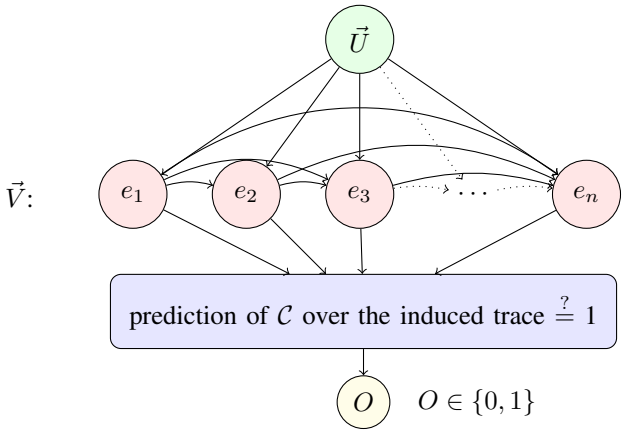

Fig. 5: Graphical representation of our causal model of processes.

IV. Computing Responsibilities for Process Outcomes

In this section, we present and analyze an algorithm that approximates the causal responsibilities of events in the process for the monitor prediction. This is based on the definitions of actual cause and degree of responsibility, as well as the causal model introduced in Sec. III. Essentially, the algorithm approximates an importance ranking over the trace events. This ranking can then be used in order to compute explanations, as we show in Section V.

A. Description of the algorithm

The responsibility approximation algorithm is presented in Alg. 1. It takes as input the trace which we aim to explain, $\sigma = \langle e_1, e_2, \ldots, e_n \rangle$, and the predictive process monitor $\mathcal{C}$, whose prediction we aim to explain. As we do not have access to the exact causal dependencies and structural equations, the algorithm approximates interventions by sampling traces similar to σ. To restrict the set of considered interventions to those that are reachable in the process, we assume access to a process simulator $\mathcal{S}$ capable of generating traces from the process. This simulator may either be provided by the user or discovered from an initial dataset of traces, $\mathcal{D}$, as shown in Sec. V. The algorithm also receives $\mathcal{D}$ and a budget k specifying the number of samples to generate.

The algorithm starts by creating k trace samples using the simulator $\mathcal{S}$ (line 3). The sampling process also uses the original trace σ to produce samples similar to σ. If computationally feasible, this step can instead enumerate all possible samples. These samples are added to $\mathcal{D}$, and the process monitor $\mathcal{C}$ is used to compute predictions for all resulting traces. Next, in line 6, the algorithm iterates over the samples and checks whether any of them changes the prediction of $\mathcal{C}$, corresponding to condition AC2 in Def. 1. When such

Algorithm 1 Explanation search algorithm

Input: A trace $\sigma = \langle e_1, e_2, \ldots, e_n \rangle$, process monitor $\mathcal{C}$, process simulator $\mathcal{S}$, dataset of traces $\mathcal{D} = \{\sigma_1, \sigma_2, \ldots, \sigma_n\}$, num of samples k

1: *origPred* ← predict_label($\mathcal{C}$,σ)
2: ▷ Sampling using the simulator
3: $\mathcal{D} \leftarrow \mathcal{D} \cup$ sample_k_traces($\mathcal{S}$,σ,k)
4: $p(\sigma_i)$ ← predict_label($\mathcal{C}$,σ_i) for σ_i in $\mathcal{D}$
5: *resp*$(e_i) \leftarrow 0$ for every e_i in σ
6: **for** σ_i in $\mathcal{D}$ **do**
7: ▷ Check if sample prediction differs from the original
8: **if** $p(\sigma_i) \neq$ *origPred* **then**
9: $S \leftarrow$ GET_CANDIDATE_CAUSE(σ, σ_i)
10: *isMinimal* ← $true$
11: **for** σ'_i in $\mathcal{D}$ **do** ▷ Checking minimality
12: $S' \leftarrow$ GET_CANDIDATE_CAUSE(σ, σ'_i)
13: **if** $S' \subset S$ and $p(\sigma'_i) \neq origPred$ **then**
14: *isMinimal* ← $false$
15: **break**
16: **if** *isMinimal* **then**
17: **for** e_i in S **do**
18: ▷ Approximating responsibility
19: *resp*$(e_i) \leftarrow \max($*resp*$(e_i), 1/(1 + |S|))$
20: **return** *resp*
21: **procedure** GET_CANDIDATE_CAUSE(original trace σ, sampled trace σ', dataset of traces $\mathcal{D}$ and their prediction p)
22: **for** i in $\{1, \ldots, |\sigma'|\}$ **do**
23: ▷ Check if prediction is guaranteed from this prefix
24: *diff* ← $\{\, \sigma'' \in \mathcal{D} \mid \sigma'[:i] = \sigma''[:i] \wedge p(\sigma') \neq p(\sigma'') \,\}$
25: **if** *diff* $= \emptyset$ **then**
26: $S \leftarrow$ get_set_of_changed_events($\sigma[:i]$,$\sigma'[:i]$)
27: **return** S

a sample is found, the algorithm approximates the minimal underlying cause from it using the "get candidate cause" procedure (defined from line 21). This procedure looks for the smallest prefix of the sampled trace from which the change in the prediction is guaranteed in $\mathcal{D}$ (lines 24-25). The subset of changed events from this prefix is the approximated candidate cause S. Starting from line 11, the algorithm then checks the minimality of S, corresponding to AC3 in Def. 1. Specifically, it searches the dataset for evidence that a strict subset of S is enough to change the prediction. If no such subset is found, S is considered a minimal cause. The algorithm then iterates over the events in S and updates their responsibility values whenever the newly computed responsibility exceeds the previously recorded value (line 19).

*a) **Computational Considerations**:* Let n be the total number of traces in $\mathcal{D}$, including the k additional sampled traces. The dominating part in terms of complexity is the nested loop starting in Line 6, which calls the "get candidate cause" procedure $O(n^2)$ times. When considering the complexity of the "get candidate cause" procedure, we get an overall complexity of $O(n^3m^2)$, where m is the maximal trace length. The algorithm makes $O(n)$ calls for prediction from $\mathcal{C}$.

B. Correctness analysis

We first prove that if given access to the causal model, the output of Alg. 1 satisfies Def. 1.

Lemma 1: If Alg. 1 has access to the causal model, its computed cause S is an actual cause of the outcome according to Def. 1.

Proof. AC1 is straightforward since it requires the causes and outcome to hold in the original setting, and indeed, we only consider the events that occurred in σ as causes of the actual predicted label. Regarding AC2, if given access to the causal model, we can identify the exact set of interventions we consider instead of approximating it using sampling. It follows that we can exactly determine the set of events such that changing them changes the outcome, hence satisfying AC2. The minimality follows a similar line of reasoning: the minimization procedure in the algorithm tries to reduce the candidate set S by checking the effect of removing events from S. If given access to the causal model, the algorithm can compute the result directly, and hence accurately minimize the subset cause S given an intervention. Since we have an accurate set of all possible interventions, we can exactly determine if such S is a minimal cause, and therefore satisfy AC3. As the causal model is not known, the algorithm instead uses a simulator to produce sample traces. The traces produced by the simulator are not necessarily for the context of the original input trace. However, if given access to all traces, we can provide correctness guarantees as shown in the following lemma.

Lemma 2: If Alg. 1 has access to all possible traces of process P, its computed cause S satisfies AC1 and AC2 in Def. 1.

Proof. AC1 follows by the same reasoning as in the previous proof, since we consider only events that actually occurred in σ as candidate causes of the actual predicted label. Regarding AC2, let σ' be the sampled trace from which S is taken, and let $\sigma'[:i]$ be the prefix at which the "get candidate cause" procedure ended its execution. $\sigma'[:i]$ contains all elements in S. Since the algorithm has access to all possible samples of P, we know that the actual continuation of $\sigma'[:i]$ under context $\vec{u}$ in the causal model is one of them. We denote this trace as σ''. Clearly, the prediction of σ'' is different from that of σ, since the algorithm checked that as a part of the "get candidate cause" procedure. Therefore, there exists an intervention that changes σ to σ'' and changes the prediction of the outcome. Hence, the algorithm satisfies AC2. Intuitively, AC2 holds because the simulator produces a *superset* of the intervention traces, as it produces the results of all interventions in all possible contexts. It includes, in particular, the results of intervention on the input context, hence the satisfaction of AC2. The minimality condition AC3 may not hold, but the algorithm attempts to approximate minimality by considering subsets of the candidate in its search procedure.

In practice, the simulator produces only a subset of all possible traces. It is, therefore, of little surprise that the algorithm produces an approximation of the actual causes. The goodness of the approximation directly depends on the exhaustiveness of the sample set, which we analyze in Sec. V. The experiments show that in practice, the output of Alg. 1 is close to optimal and is more accurate than that of other methods.

The role of $\vec{W}$*:* In Def. 1, the set $\vec{W}$ is a set of variables that are *clamped* to their original values when checking the effect of intervening on $\vec{X}$. Recall that our mechanism for observing the results of interventions is examining the output of a given simulator, which is a probability distribution over the sequence of events. Therefore, it is impossible to measure the precise effect of an intervention or the clamping of a variable. Moreover, clamping the values of variables can result in an unreachable trace. To ensure reachability, the responsibility formula in Alg. 1 assumes that $\vec{W} = \emptyset$. Indeed, as we have only one main causal path (see Fig. 5), if we intervene on the event e_i, then clamping any of the events before it is meaningless (they have their original values anyway), and actively clamping any of the events after e_i could interrupt the propagation of the change or result in an unreachable trace. This is illustrated in Fig. 3. This restriction may lead to an overapproximation of some responsibility values. However, as the lemmas show, the resulting algorithm captures actual causes. Furthermore, as demonstrated in Section V, the resulting responsibility ranking is highly effective in practice for computing explanations.

V. Experimental Evaluation

This section evaluates our proposed approach for generating explanations in several case studies. To measure the quality of an explanation, we adopt the well-known insertion test [18] which derives a *minimal sufficient* set of features based on their importance (responsibility) ranking, provided by the explanation algorithm. Specifically, features are greedily added based on their order of importance until the selected subset alone is sufficient to reproduce the prediction of the explained instance. That means that sufficiency is satisfied if all instances in the dataset that share the same valuation on this subset receive the same classifier prediction as the explained instance. The size of this sufficient subset serves as a proxy for how accurate the ranking is. Following several principles presented in [16], [18], [19], our evaluation concentrates on three main criteria for explanations: 1) Consistency: the explanation algorithm should output similar explanations when executed several times, 2) Efficiency: the explanation algorithm should be efficient and produce an explanation in a reasonable time, and 3) Succinctness: the explanation produced by the algorithm should be as concise as possible.

A. Case Studies

We evaluated our approach on a predictive process monitoring benchmark presented in [20]. This benchmark is based on nine real-life event logs, eight of which are publicly available and one of which is private. The public logs can be accessed through the 4TU Centre for Research Data[1]. From these logs, [20] constructed 22 public case study datasets with binary target labels. Some of the datasets originate from the same underlying event log but differ in their labeling. That is, some datasets represent the same processes but with different outcomes that the monitor is tasked with predicting. Table I summarizes the main statistics of these datasets, including the number of traces and variants, trace length, number of activity classes, and the label distribution. The table also reports the predictive accuracy of the classifiers used in our experiments. Each dataset was split into 80% train and 20% test.

B. Implementation

For the evaluation, we implemented Alg. 1 in the AC4PM tool. The process simulator used is a Long Short-Term Memory (LSTM) neural network implemented using PyTorch [21] and trained to predict the next event given a trace prefix. We pretrained a separate simulator for each case study using its corresponding train dataset. The algorithm input dataset, $\mathcal{D}$, was the train set, and we used $k = 100$ additional samples. The complete code for the evaluation and the tool are available at https://figshare.com/s/7d9602ca44991641c56a.

C. Comparison

To assess our contribution, we compared our approach with the current state-of-the-art of local explainability tools for PPM: LIME [4] and SHAP [3]. Both methods assign importance scores to features, assessing their contribution to the prediction, which aligns with our proposed causal degree of responsibility ranking. SHAP offers both model-dependent and model-agnostic variants. Since our setting (as well as LIME) assumes a black-box classifier, and we evaluate two different types of classifiers, we selected KernelSHAP to ensure a fair and consistent comparison.

D. Experimental Setup

The training set was used to train two predictive model classifiers, for which we derived explanations: eXtreme Gradient Boosting (XGBoost) [22] and a Multi-layer Perceptron (MLP) neural network implemented using PyTorch [21]. We used a standard static one-hot encoder for the dataset encoding. The performance of the trained models over the test set is also reported in Table I. For each classifier, we compared the results of AC4PM with those of LIME and SHAP. For the evaluation, we sampled 10 traces from the test set, on which we ran the explanation algorithms. To reduce the effect of poor predictions, we only took samples that the model predicted correctly, that is, true negatives or true positives. No restrictions were placed on trace length, variant frequency, or any other trace characteristics during sampling. For each case, we ran the explanation algorithms 10 times with different random seeds. Experiments were conducted on an Apple M4 Max machine with 32GB RAM.

[1] https://www.4tu.nl/htm/research/4tu-research-data

TABLE I: Statistics of the datasets used for the experiments.

dataset	#traces	#variants	min length	max length	mean length	#event classes	pos class ratio	test accuracy mlp	test accuracy xgboost
traffic_fines_1	129615	200	2	20	8.12	10	0.13	0.92	0.9
sepsis_cases_2	782	707	4	60	14.38	15	0.11	0.97	0.93
sepsis_cases_4	782	709	4	185	16.54	15	0.84	0.93	0.75
sepsis_cases_1	782	709	5	185	17.36	15	0.15	0.73	0.77
hospital_3	77525	542	2	217	10.59	17	0.21	0.87	0.85
hospital_2	77525	1005	2	217	12.39	18	0.13	0.96	0.96
Production	220	203	1	78	12.0	26	0.52	0.78	0.76
BPIC17_O_Cancelled	31413	15846	10	180	48.44	26	0.41	0.99	0.96
BPIC17_O_Refused	31413	15846	10	180	48.44	26	0.15	1.0	0.99
BPIC17_O_Accepted	31413	15846	10	180	48.44	26	0.44	0.99	0.95
bpic2012_O_DECLINED	4685	3790	15	175	43.44	36	0.17	0.99	0.97
bpic2012_O_ACCEPTED	4685	3790	15	175	43.44	36	0.54	0.99	0.93
bpic2012_O_CANCELLED	4685	3790	15	175	43.44	36	0.28	0.99	0.93
BPIC11_f3	1121	811	1	1368	83.15	190	0.18	0.94	0.91
BPIC11_f1	1140	816	1	1814	77.26	193	0.36	0.87	0.85
BPIC11_f4	1140	978	1	1432	94.5	231	0.33	0.86	0.8
BPIC11_f2	1140	978	1	1814	152.44	251	0.75	0.82	0.86
BPIC15_4_f2	577	576	1	82	42.07	319	0.16	0.98	0.96
BPIC15_5_f2	1051	1049	5	134	52.0	376	0.31	0.99	0.98
BPIC15_1_f2	696	677	2	101	42.22	380	0.23	0.93	0.94
BPIC15_3_f2	1328	1285	3	124	44.47	380	0.2	0.95	0.98
BPIC15_2_f2	753	752	1	132	54.77	396	0.19	0.95	0.95

E. Evaluation Metrics

Based on the evaluation criteria defined above, we assessed the explanation methods using the following metrics. To measure consistency, we computed the Variable Stability Index (VSI) [23], which quantifies the similarity between explanations output by the insertion tests across multiple runs. For efficiency, we measured the average time required to generate an explanation for a single trace. To evaluate succinctness, we measure the fraction of the explanation out of the total trace.

F. Results

The results of the local explanation tools for the MLP and the XGBoost classifier are reported in Table II and Table III, respectively. The tables present the VSI metric, the explanation size, presented as the fraction of trace elements that appear in the explanation generated using the insertion test, and the average execution time of the explanation algorithm in seconds. AC4PM and LIME successfully completed their runs across all the evaluated datasets. However, SHAP ran out of memory (*o.m.*) in some settings, even when configured with parameters expected to significantly reduce memory consumption.

For the VSI metric, AC4PM achieves the highest stability in the majority of datasets across both classifiers. In the case of XGBoost, LIME appears slightly more stable than AC4PM. However, we argue that this effect is mainly due to variability in LIME's performance rather than a decrease in the stability of AC4PM: when computing Spearman's correlation [24] between the VSI scores of each method across the two classifiers, LIME shows a relatively low correlation ($\rho \approx 0.43$), whereas AC4PM exhibits a much higher correlation ($\rho \approx 0.88$). Importantly, the classifiers themselves did exhibit significant differences in predictive performance, as reflected by their test accuracies in Table I. This suggests that, although LIME is model-agnostic, its stability is highly affected by the type of classifier used, whereas AC4PM (and SHAP) are more consistent.

Another interesting aspect is the effect of dataset attributes on the stability of the explanation algorithms. We observed that the stability of AC4PM generally increases with the number of trace variants in the dataset, with moderate positive Spearman correlations between these variables (MLP: $\rho \approx 0.44$, XGBoost: $\rho \approx 0.49$). This is largely expected, as more data reduces the variability introduced by additional intervention samples. A similar trend can be observed for SHAP. In contrast, the stability of LIME generally decreases as the number of variants grows (MLP $\rho \approx -0.32$, XGBoost $\rho \approx -0.09$). A similar pattern appears when examining the effect of trace length. The stability of AC4PM increases with the mean trace length (MLP $\rho \approx 0.22$, XGBoost $\rho \approx 0.18$), whereas the stability of LIME decreases (MLP $\rho \approx -0.66$, XGBoost $\rho \approx -0.16$). The poor stability of LIME is expected. LIME explains predictions by fitting a local surrogate model to randomly generated perturbations of the input. In PPM, the dimensionality of the one-hot encoded representation grows with both the number of activities and the trace length, exacerbating the curse of dimensionality. As a result, the surrogate model becomes increasingly sensitive to the specific perturbation samples used, leading to lower explanation stability [23].

The explanation size measures the fraction of trace elements sufficient to generate the prediction, and thus reflects how succinct the explanation is. Across the evaluated datasets, explanations derived using AC4PM are consistently smaller than those produced by LIME and SHAP. This suggests that AC4PM tends to identify a more focused subset of events that is sufficient to reproduce the prediction. Further, the size of AC4PM explanations appears to *decrease* as process complexity increases, both with respect to the number of activities (MLP $\rho \approx -0.71$, XGBoost $\rho \approx -0.68$) and the mean trace

TABLE II: MLP classifier explanation results

	VSI			Explanation size[1]			Execution time[2]		
Dataset	**AC4PM**	**LIME**	**SHAP**	**AC4PM**	**LIME**	**SHAP**	**AC4PM**	**LIME**	**SHAP**
traffic_fines_1	**0.99**	0.89	0.93	**0.15**	0.22	0.22	**0.03**	0.06	3.39
sepsis_cases_2	0.91	0.54	**1.0**	0.22	0.13	**0.07**	**0.22**	4.43	2.43
sepsis_cases_4	0.89	**0.96**	0.78	**0.26**	0.43	0.37	**1.13**	2.33	2.19
sepsis_cases_1	0.77	0.63	**0.9**	0.23	0.4	**0.07**	0.97	**0.62**	1.73
hospital_3	0.86	0.82	**0.97**	**0.2**	0.47	0.39	**1.17**	3.81	2.69
hospital_2	0.8	**0.92**	0.84	0.27	0.83	**0.25**	1.22	**0.85**	2.63
Production	0.82	0.78	**0.91**	0.18	0.53	**0.17**	**0.27**	0.76	0.79
BPIC17_O_Cancelled	**0.99**	0.85	*o.m.*	**0.33**	0.63	*o.m.*	83.75	**1.85**	*o.m.*
BPIC17_O_Refused	**0.98**	0.75	*o.m.*	**0.16**	0.63	*o.m.*	55.17	**2.55**	*o.m.*
BPIC17_O_Accepted	**0.99**	0.53	*o.m.*	**0.12**	0.32	*o.m.*	83.65	**2.25**	*o.m.*
bpic2012_O_DECLINED	**0.97**	0.76	*o.m.*	**0.18**	0.61	*o.m.*	11.57	**2.13**	*o.m.*
bpic2012_O_ACCEPTED	**0.96**	0.44	*o.m.*	**0.27**	0.37	*o.m.*	26.82	**2.15**	*o.m.*
bpic2012_O_CANCELLED	**0.98**	0.63	*o.m.*	**0.17**	0.5	*o.m.*	15.64	**2.26**	*o.m.*
BPIC11_f3	**0.9**	0.36	*o.m.*	**0.04**	0.31	*o.m.*	191.19	**19.52**	*o.m.*
BPIC11_f1	**0.93**	0.56	*o.m.*	**0.1**	0.3	*o.m.*	321.35	**16.97**	*o.m.*
BPIC11_f4	**0.96**	0.41	*o.m.*	**0.15**	0.25	*o.m.*	312.27	**21.36**	*o.m.*
BPIC11_f2	**1.0**	0.28	*o.m.*	**0.02**	0.24	*o.m.*	571.42	**28.71**	*o.m.*
BPIC15_4_f2	0.72	**0.81**	0.23	**0.06**	0.83	0.11	**1.16**	3.43	22.59
BPIC15_5_f2	**0.73**	0.59	*o.m.*	**0.07**	0.65	*o.m.*	5.87	**5.38**	*o.m.*
BPIC15_1_f2	0.8	**0.83**	0.34	**0.09**	0.89	0.14	**1.74**	4.65	47.63
BPIC15_3_f2	**0.81**	0.7	*o.m.*	**0.04**	0.77	*o.m.*	6.86	**5.22**	*o.m.*
BPIC15_2_f2	0.7	**0.77**	0.28	**0.11**	0.76	0.2	**4.42**	5.79	78.35

[1] fraction out of the total trace, [2] in seconds, *o.m.* out of memory

length (MLP $\rho \approx -0.53$, XGBoost $\rho \approx -0.52$). This suggests that AC4PM produces more focused explanations in processes with greater variability. In such settings, predictions are likely driven by more specific patterns, allowing AC4PM to isolate smaller sets of causal events. In contrast, we did not observe a similar pattern for LIME and SHAP, whose correlations did not indicate a consistent direction.

Across the evaluated datasets, the runtime of all methods increases with the complexity of the underlying process data, particularly with respect to the number of traces, their length, and the overall number of activities. This especially affected the execution of AC4PM, whose runtime grows substantially as traces become longer and the number of traces increases. This behavior is consistent with the complexity analysis presented in Section IV. Despite this dependency, the explanation algorithms generally remain relatively efficient, producing explanations within a few minutes across all evaluated benchmarks. That said, the current, proof-of-concept implementation of the AC4PM approach has considerable room for optimization. We therefore expect that the runtime of the algorithms can be significantly reduced in the future.

Different aspects of dataset complexity also affected the runtime of the compared explanation methods. LIME appeared particularly sensitive to longer traces, whereas SHAP was more strongly affected by datasets with a larger number of distinct activities. This behavior is expected, as both trace length and the number of activities directly influence the dimensionality of the one-hot encoding, substantially increasing the computational cost of these methods.

VI. Related Work

Several studies have explored the use of explainable AI techniques in the context of process mining. Some have investigated the applicability of general explanation tools such as LIME and SHAP for interpreting predictions of PPM models and understanding the influence of process attributes and activities on outcomes [2], [6], [19]. These approaches typically provide feature-based explanations but often treat process traces in the same format as they were fed into the PPM model, usually in the form of tabular data, ignoring the temporal and structural dependencies processes exhibit.

Another line of work investigates counterfactual explanations for PPM predictions [25], [26]. A more recent work has explored counterfactual explanations for process predictions, highlighting several important features of processes in the context of counterfactual trace generation, such as temporal ordering [11], [27]. Similarly, [28] propose a framework for case-level counterfactual reasoning over event logs using structural equation models. While counterfactual explanations may provide valuable insights in some cases, they do not accurately capture causal relationships in more complex cases, as discussed in the introduction.

More broadly, several approaches have explored the use of causal analysis in process mining [29]–[34]. These works primarily focus on global (process-level) causal discovery and analysis, rather than local (case-level) explanations for predictions of black-box PPM models, which is the focus of this paper.

Actual causality is well-suited for providing explanations for events that already occurred [8]. It has been used to explain AI models across various domains, such as images [16], audio [35], vibrational spectroscopy [36], and failures of autonomous cyber-physical systems [37]. These methods approximate causal responsibility to efficiently find causal explanations of different sizes, confidences, and multiplicities [38],

TABLE III: XGBoost classifier explanation results

	VSI			Explanation size[1]			Execution time[2]		
Dataset	**AC4PM**	**LIME**	**SHAP**	**AC4PM**	**LIME**	**SHAP**	**AC4PM**	**LIME**	**SHAP**
traffic_fines_1	**0.98**	0.97	**0.98**	**0.16**	0.25	0.18	**0.03**	0.04	0.9
sepsis_cases_2	0.86	0.64	**1**	0.22	0.08	**0.07**	**0.21**	0.85	3.24
sepsis_cases_4	0.8	0.74	**0.98**	0.17	**0.13**	**0.13**	**1.33**	1.92	4.26
sepsis_cases_1	0.86	0.75	**1**	0.12	0.29	**0.09**	**1.06**	1.36	5.98
hospital_3	0.82	0.89	**0.98**	**0.19**	0.5	0.4	**1.16**	1.76	2.91
hospital_2	0.83	**0.95**	0.92	0.27	0.78	**0.21**	**1.3**	1.72	7.24
Production	0.83	**0.97**	0.94	0.16	0.45	**0.1**	**0.28**	1.35	3.79
BPIC17_O_Cancelled	**0.99**	0.93	*o.m.*	**0.28**	0.51	*o.m.*	83.01	**3.79**	*o.m.*
BPIC17_O_Refused	**0.97**	0.88	*o.m.*	**0.16**	0.53	*o.m.*	55.4	**3.95**	*o.m.*
BPIC17_O_Accepted	**0.99**	0.72	*o.m.*	**0.13**	0.45	*o.m.*	88.04	**7.7**	*o.m.*
bpic2012_O_DECLINED	0.92	**0.98**	*o.m.*	**0.13**	0.43	*o.m.*	11.77	**3.24**	*o.m.*
bpic2012_O_ACCEPTED	**0.99**	0.78	*o.m.*	0.26	**0.12**	*o.m.*	25.89	**2.98**	*o.m.*
bpic2012_O_CANCELLED	**0.98**	0.91	*o.m.*	**0.16**	0.39	*o.m.*	15.39	**3.31**	*o.m.*
BPIC11_f3	0.92	**0.93**	*o.m.*	**0.04**	0.18	*o.m.*	191.41	**26.3**	*o.m.*
BPIC11_f1	0.86	**0.88**	*o.m.*	**0.14**	0.38	*o.m.*	348.04	**23.02**	*o.m.*
BPIC11_f4	**0.95**	0.84	*o.m.*	**0.13**	0.24	*o.m.*	232.75	**29.34**	*o.m.*
BPIC11_f2	**0.97**	0.85	*o.m.*	**0.03**	0.1	*o.m.*	680.93	**40.55**	*o.m.*
BPIC15_4_f2	0.79	**0.92**	0.77	**0.06**	0.17	0.1	**1**	4.99	49.61
BPIC15_5_f2	0.74	**0.85**	*o.m.*	**0.07**	0.21	*o.m.*	**5.9**	8.73	*o.m.*
BPIC15_1_f2	0.75	**0.85**	0.8	0.12	0.34	**0.11**	**2.64**	6.74	94.87
BPIC15_3_f2	0.79	**0.94**	*o.m.*	**0.04**	0.26	*o.m.*	**6.88**	7.72	*o.m.*
BPIC15_2_f2	0.79	0.94	**0.97**	0.08	0.3	**0.04**	**3.94**	8.81	216.64

[1] fraction out of the total trace, [2] in seconds, *o.m.* out of memory

[39]. These papers assume causal independence between the inputs and the AI model as a black box, and work with depth-2 causal models. In contrast, in this work, we take into account the causal dependencies between the variables, hence our causal models have a higher depth.

VII. Conclusion

In this paper, we presented a novel approach for generating local explanations for predictions produced by Predictive Process Monitoring (PPM) models using the framework of actual causality. We introduced a causal model tailored to processes that captures the temporal dependencies between events in the trace and enables reasoning about their influence over the predicted outcome. Using this model, we proposed an algorithm that approximates the causal responsibility of events in a trace in order to estimate their importance to the predicted outcome. We evaluated the algorithm on a range of datasets from a widely used PPM benchmark and compared it with state-of-the-art local explanation approaches. The experimental results demonstrate that our proposed approach produces explanations that are generally more stable and concise, while remaining computationally practical. Moreover, for larger sample sets, the explanations computed by our algorithms are more precise, and the approach is stable wrt the complexity of the traces.

As this is the first step towards applying the framework of actual causality to explain PPM models, we focused on a simplified representation of process traces, where each event is described by its activity name and timestamps are used solely for event ordering. While this setting suits many use cases, processes often include richer information, such as continuous process attributes or contextual event data. Extending the framework to support richer process representations is therefore an important direction for future work. We believe that such an extension is feasible, as previous work has successfully applied actual causality to explain an AI system operating in a high-dimensional input space [35], [36]. Future research will also focus on algorithmic solutions that account for the size of the contingency set when approximating responsibility. This will enable more accurate estimates of responsibility and more faithful rankings.

References

[1] P. Ceravolo, M. Comuzzi, J. De Weerdt, C. Di Francescomarino, and F. Maggi, "Predictive process monitoring: concepts, challenges, and future research directions," *Process Science*, vol. 1, no. 1, p. 2, Sep. 2024. [Online]. Available: https://doi.org/10.1007/s44311-024-00002-4

[2] W. Rizzi, M. Comuzzi, C. Di Francescomarino, C. Ghidini, S. Lee, F. Maggi, and A. Nolte, "Explainable predictive process monitoring: a user evaluation," *Process Science*, vol. 1, no. 1, p. 3, Oct. 2024. [Online]. Available: https://doi.org/10.1007/s44311-024-00003-3

[3] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1135–1144. [Online]. Available: https://doi.org/10.1145/2939672.2939778

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

[5] N. Kim, H. Wittges, and S. Rinderle-Ma, "Illuminating Predictions: The Use and Evaluation of Explainable AI in Predictive Process Monitoring – A Systematic Review," *Procedia Computer Science*, vol. 270, pp. 1149–1158, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050925029060

[6] G. Elkhawaga, M. Abu-Elkheir, and M. Reichert, "Explainability of Predictive Process Monitoring Results: Can You See My Data Issues?" *Applied Sciences*, vol. 12, no. 16, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/16/8192

[7] J. Y. Halpern and J. Pearl, "Causes and Explanations: A Structural-Model Approach. Part I: Causes," *The British Journal for the Philosophy of Science*, vol. 56, no. 4, pp. 843–887, Dec. 2005. [Online]. Available: https://www.journals.uchicago.edu/doi/abs/10.1093/bjps/axi147

[8] J. Y. Halpern, *Actual Causality*. The MIT Press, 2019.

[9] J. Pearl, *Causality*. Cambridge University Press, Sep. 2009, google-Books-ID: f4nuexsNVZIC.

[10] N. Mehdiyev, M. Majlatow, and P. Fettke, "Interpretable and explainable machine learning methods for predictive process monitoring: a systematic literature review," *Artificial Intelligence Review*, vol. 58, no. 12, p. 378, Oct. 2025. [Online]. Available: https://doi.org/10.1007/s10462-025-11399-0

[11] A. Buliga, C. Di Francescomarino, C. Ghidini, M. Montali, and M. Ronzani, "Generating Counterfactual Explanations Under Temporal Constraints," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 15, pp. 15 622–15 631, Apr. 2025. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/33715

[12] C. Di Francescomarino and C. Ghidini, *Predictive Process Monitoring*. Cham: Springer International Publishing, 2022, pp. 320–346. [Online]. Available: https://doi.org/10.1007/978-3-031-08848-3_10

[13] J. Y. Halpern, "A modification of the Halpern–Pearl definition of causality," in *Proceedings of the International Joint Conferences on Artificial Intelligence*. AAAI Press, 2015, pp. 3022–3033.

[14] H. Chockler and J. Y. Halpern, "Responsibility and Blame: A Structural-Model Approach," *Journal of Artificial Intelligence Research*, vol. 22, pp. 93–115, Oct. 2004. [Online]. Available: https://www.jair.org/index.php/jair/article/view/10386

[15] ——, "Explaining image classifiers," in *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR*, 2024.

[16] H. Chockler, D. A. Kelly, D. Kroening, and Y. Sun, "Causal explanations for image classifiers," *Journal of Artificial Intelligence Research*, 2026.

[17] J. Y. Halpern, "Defaults and normality in causal structures," in *Proceedings of KR*, ser. KR'08. AAAI Press, 2008, p. 198–208.

[18] N. Hama, M. Mase, and A. B. Owen, "Deletion and Insertion Tests in Regression Models," *Journal of Machine Learning Research*, vol. 24, no. 290, pp. 1–38, 2023. [Online]. Available: http://jmlr.org/papers/v24/22-0560.html

[19] G. El-khawaga, M. Abu-Elkheir, and M. Reichert, "XAI in the Context of Predictive Process Monitoring: An Empirical Analysis Framework," *Algorithms*, vol. 15, no. 6, 2022. [Online]. Available: https://www.mdpi.com/1999-4893/15/6/199

[20] I. Teinemaa, M. Dumas, M. L. Rosa, and F. M. Maggi, "Outcome-Oriented Predictive Process Monitoring: Review and Benchmark," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 2, Mar. 2019. [Online]. Available: https://doi.org/10.1145/3301300

[21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d. Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[22] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785

[23] G. Visani, E. Bagli, F. Chesani, A. Poluzzi, and D. Capuzzo, "Statistical stability indices for LIME: Obtaining reliable explanations for machine learning models," *Journal of the Operational Research Society*, vol. 73, no. 1, pp. 91–101, 2022, _eprint: https://doi.org/10.1080/01605682.2020.1865846. [Online]. Available: https://doi.org/10.1080/01605682.2020.1865846

[24] P. Schober, C. Boer, and L. A. Schwarte, "Correlation Coefficients: Appropriate Use and Interpretation," *Anesthesia & Analgesia*, vol. 126, no. 5, p. 1763, May 2018. [Online]. Available: https://journals.lww.com/anesthesia-analgesia/fulltext/2018/05000/Correlation_CoefficientsAppropriate_Use_and.50.aspx

[25] T.-H. Huang, A. Metzger, and K. Pohl, "Counterfactual Explanations for Predictive Business Process Monitoring," in *Information Systems*, M. Themistocleous and M. Papadaki, Eds. Cham: Springer International Publishing, 2022, pp. 399–413.

[26] C. Hsieh, C. Moreira, and C. Ouyang, "Dice4el: Interpreting process predictions using a milestone-aware counterfactual approach," in *2021 3rd International Conference on Process Mining (ICPM)*, 2021, pp. 88–95.

[27] A. Buliga, C. Di Francescomarino, C. Ghidini, I. Donadello, and F. M. Maggi, "Guiding the generation of counterfactual explanations through temporal background knowledge for predictive process monitoring," *Data Mining and Knowledge Discovery*, vol. 39, no. 5, p. 63, Aug. 2025. [Online]. Available: https://doi.org/10.1007/s10618-025-01117-3

[28] M. S. Qafari and W. M. P. van der Aalst, "Case Level Counterfactual Reasoning in Process Mining," in *Intelligent Information Systems*, S. Nurcan and A. Korthaus, Eds. Cham: Springer International Publishing, 2021, pp. 55–63.

[29] B. F. A. Hompes, A. Maaradji, M. La Rosa, M. Dumas, J. C. A. M. Buijs, and W. M. P. van der Aalst, "Discovering Causal Factors Explaining Business Process Performance Variation," in *Advanced Information Systems Engineering*, E. Dubois and K. Pohl, Eds. Cham: Springer International Publishing, 2017, pp. 177–192.

[30] S. J. J. Leemans and N. Tax, "Causal Reasoning over Control-Flow Decisions in Process Models," in *Advanced Information Systems Engineering*, X. Franch, G. Poels, F. Gailly, and M. Snoeck, Eds. Cham: Springer International Publishing, 2022, pp. 183–200.

[31] G. V. Houdt, N. Martin, and B. Depaire, "AITIA-PM: Discovering the true causes of events in a process mining context," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 107145, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197623013295

[32] P. Waibel, L. Pfahlsberger, K. Revoredo, and J. Mendling, "Causal Process Mining from Relational Databases with Domain Knowledge," Jul. 2023, arXiv:2202.08314 [cs.DB]. [Online]. Available: http://arxiv.org/abs/2202.08314

[33] F. Fournier, L. Limonad, I. Skarbovsky, and Y. David, "The why in business processes: Discovery of causal execution dependencies," *KI-Künstliche Intelligenz*, vol. 39, no. 3, pp. 197–219, 2025.

[34] M. S. Qafari and W. van der Aalst, "Root cause analysis in process mining using structural equation models," in *Business Process Management Workshops*, A. Del Río Ortega, H. Leopold, and F. M. Santoro, Eds. Cham: Springer International Publishing, 2020, pp. 155–167.

[35] D. A. Kelly and H. Chockler, "I Guess That's Why They Call it the Blues: Causal Analysis for Audio Classifiers," Jan. 2026, arXiv:2601.16675 [cs.SD]. [Online]. Available: http://arxiv.org/abs/2601.16675

[36] N. Blake, D. A. Kelly, A. Chanchal, S. Kapllani-Mucaj, G. Thomas, and H. Chockler, "SpecReX: Explainable AI for Raman Spectroscopy," Mar. 2025, arXiv:2503.14567 [cs.LG]. [Online]. Available: http://arxiv.org/abs/2503.14567

[37] K. Elimelech, T. Yaacov, D. A. Kelly, H. Chockler, and M. Y. Vardi, "Explaining failures of cyber-physical systems with actual causality," in *International Conference on Robotics and Automation (ICRA)*. IEEE, 2026.

[38] H. Chockler, D. A. Kelly, and D. Kroening, "Multiple different explanations for image classifiers," in *ECAI European Conference on Artificial Intelligence*, 2025.

[39] D. A. Kelly, A. Chanchal, and N. Blake, "I Am Big, You Are Little; I Am Right, You Are Wrong," 2025, pp. 817–826. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2025/html/Kelly_I_Am_Big_You_Are_Little_I_Am_Right_You_ICCV_2025_paper.html